

EXPERIMENT - 7

Aim: Password Cracking with John the Ripper.

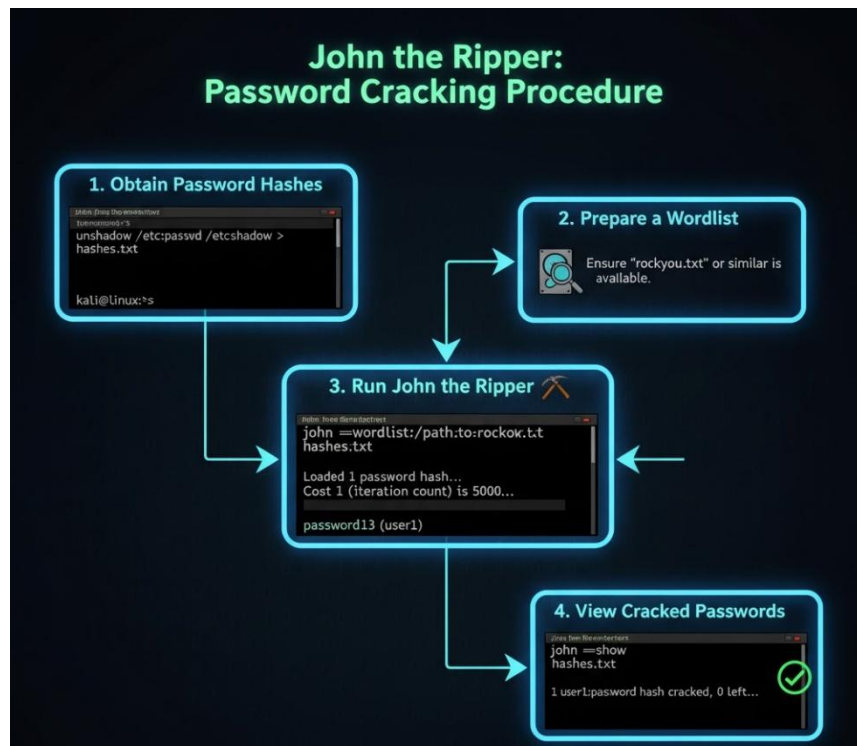
Material Required:

- John the Ripper software
- A Linux-based system (like Kali Linux) or Windows with John the Ripper installed
- A password hash file (e.g., a sample /etc/shadow file)
- A wordlist file (e.g., rockyou.txt)

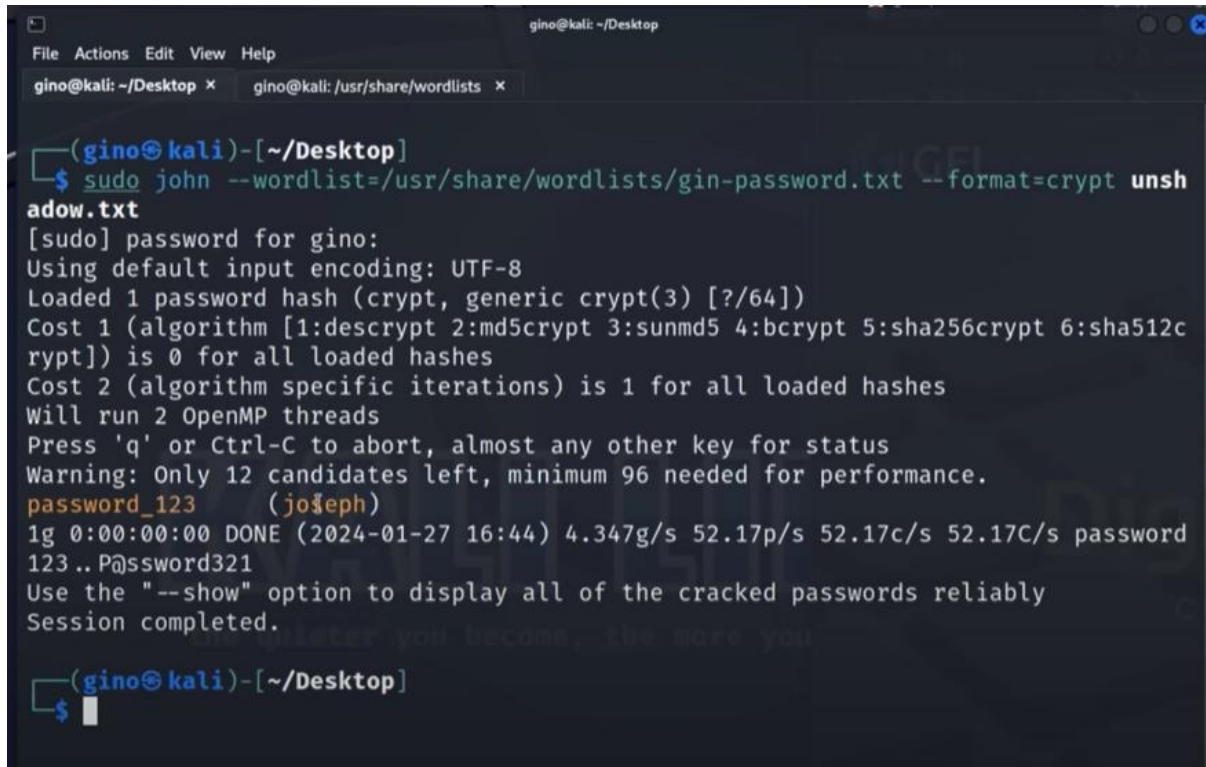
Theory: John the Ripper is a popular open-source password security auditing and password recovery tool available for many operating systems. Its primary purpose is to detect weak passwords by attempting to crack encrypted password hashes. It uses various attack methods, including:

- **Dictionary Attack:** It uses a list of common words or previously compromised passwords (a wordlist) to guess the password.
- **Brute-Force Attack:** It systematically tries every possible combination of characters until the correct password is found.
- **Hybrid Attack:** It combines dictionary attacks with brute-force by adding numbers, symbols, or altering capitalization in the words from a wordlist.

Procedure:



Observations:



```
gino@kali: ~/Desktop
File Actions Edit View Help
gino@kali: ~/Desktop x gino@kali: /usr/share/wordlists x

(gino@kali)-[~/Desktop]
$ sudo john --wordlist=/usr/share/wordlists/gin-password.txt --format=crypt unshadow.txt
[sudo] password for gino:
Using default input encoding: UTF-8
Loaded 1 password hash (crypt, generic crypt(3) [?/64])
Cost 1 (algorithm [1:descrypt 2:md5crypt 3:sunmd5 4:bcrypt 5:sha256crypt 6:sha512crypt]) is 0 for all loaded hashes
Cost 2 (algorithm specific iterations) is 1 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Warning: Only 12 candidates left, minimum 96 needed for performance.
password_123 (joseph)
1g 0:00:00:00 DONE (2024-01-27 16:44) 4.347g/s 52.17p/s 52.17c/s 52.17C/s password_123..P@ssword321
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

(gino@kali)-[~/Desktop]
$
```

Result:

John the Ripper successfully cracked the weak password from the hash file using the provided wordlist. The experiment demonstrates that passwords based on common words or simple patterns are highly vulnerable to dictionary attacks, confirming the tool's effectiveness in security auditing.

Precautions:

- **Ethical Use:** Only perform password cracking on systems and accounts for which you have explicit, written authorization. Unauthorized access is illegal.
- **System Performance:** Password cracking is a resource-intensive process. Avoid running it on critical production systems where it could impact performance.
- **Secure Wordlists:** Ensure that any wordlists used do not contain sensitive information.
- **Protect Hash Files:** The password hash file contains sensitive information. It should be handled securely and deleted after the experiment is complete.

EXPERIMENT - 8

Aim: Web traffic analysis using wireshark.

Material Required:

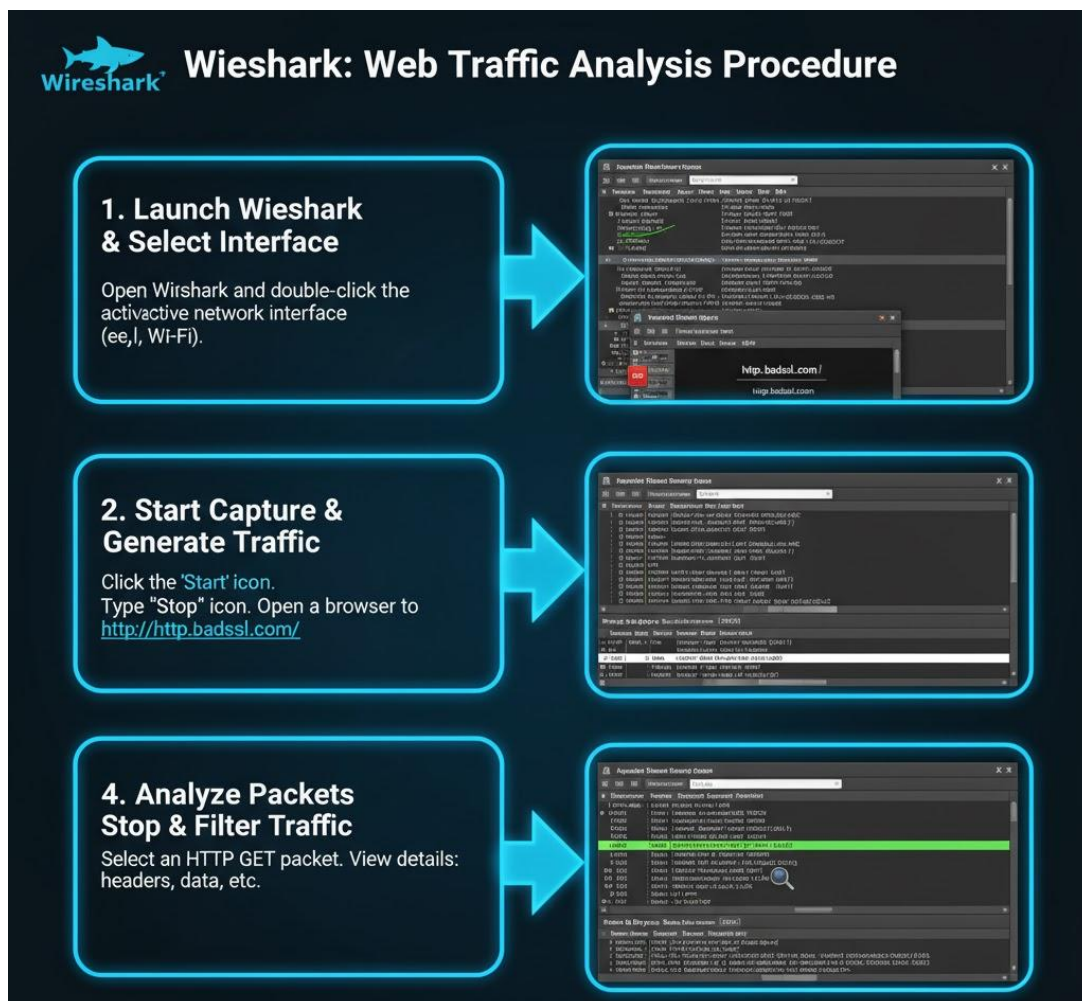
- Wireshark software
- A computer with an active internet connection
- A web browser (e.g., Chrome, Firefox)

Theory: Wireshark is a powerful and widely-used open-source network protocol analyzer. It allows users to "see" what's happening on their network at a microscopic level by capturing and displaying data packets traveling across the network in real-time.

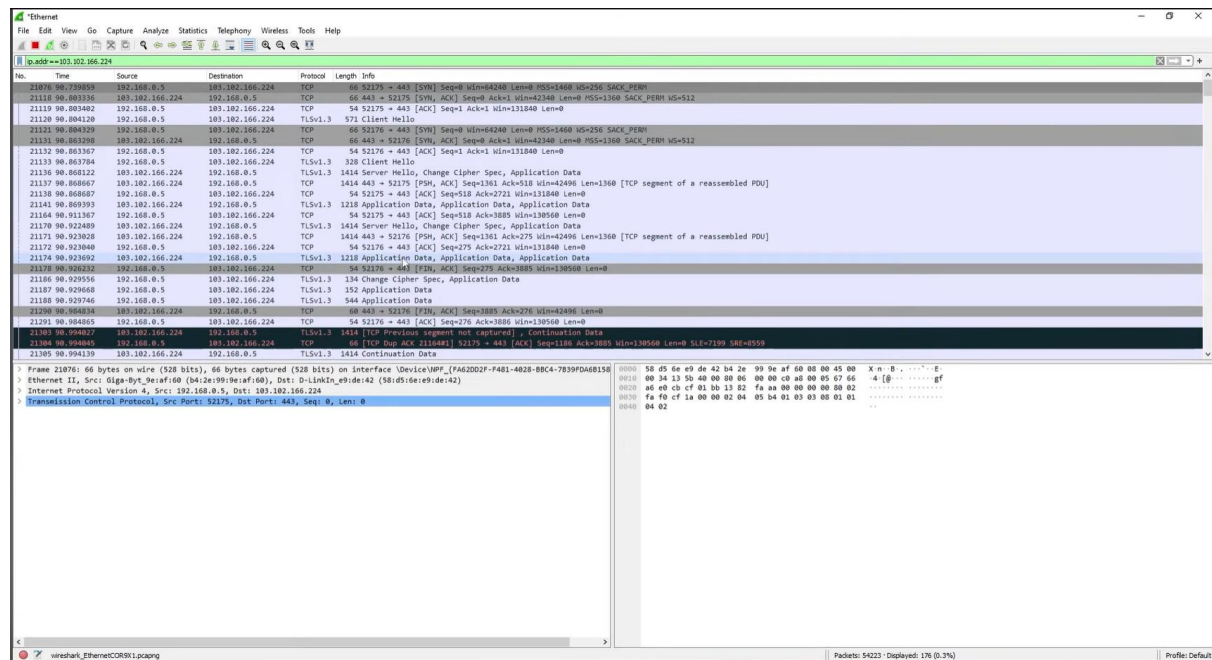
Web traffic primarily uses the **Hypertext Transfer Protocol (HTTP)** and its secure version, **HTTPS**. When you visit a website, your browser sends an HTTP/S request to the webserver, which then sends back an HTTP/S response containing the website's content (HTML, images, etc.). This entire exchange happens over the **Transmission Control Protocol/Internet Protocol (TCP/IP)** suite.

This experiment will demonstrate how to isolate and inspect these HTTP packets to understand the flow of information, including requests, responses, and the data being transferred.

Procedure:



Observations:



Result:

Wireshark successfully captured and allowed for the detailed analysis of web traffic. The experiment demonstrated the fundamental request-response model of the HTTP protocol. It also highlighted a critical security concept: unencrypted HTTP traffic is fully exposed and can be easily intercepted and read on a network, whereas HTTPS provides confidentiality by encrypting the data payload.

Precautions:

- **Ethical Use:** Only capture traffic on networks you own or have explicit permission to monitor. Unauthorized packet sniffing is a violation of privacy and is illegal in many places.
- **Select the Correct Interface:** Ensure you are capturing on the active network connection (Wi-Fi or Ethernet); otherwise, you will not see any relevant traffic.
- **Handle Data Responsibly:** Packet captures can contain sensitive information like usernames, passwords (on unencrypted sites), and personal data. Store and handle capture files securely.
- **Resource Usage:** On a very busy network, a continuous packet capture can consume a significant amount of memory and disk space. Stop the capture as soon as you have the data you need.

EXPERIMENT - 9

Aim: To identify phishing URLs using online and browser tools.

Material Required:

- A modern web browser (e.g., Chrome, Firefox, Edge).
- An online URL analysis tool (e.g., VirusTotal, Google Safe Browsing).
- A list of sample URLs to test (including known safe URLs and suspected phishing URLs).

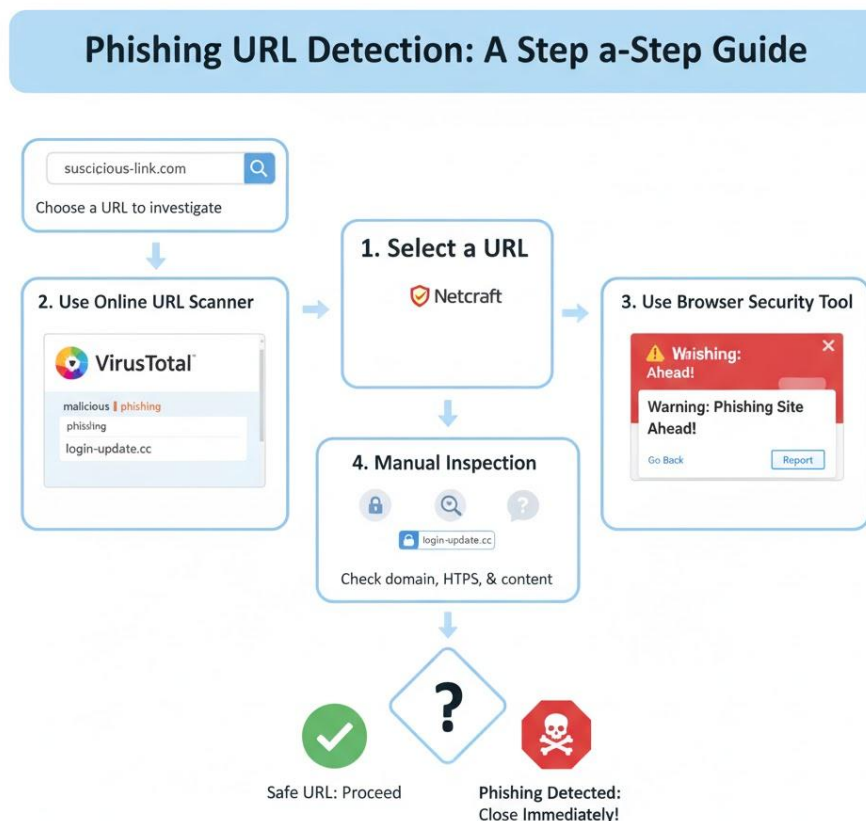
Theory:

Phishing is a cyberattack where attackers impersonate legitimate organizations to trick individuals into revealing sensitive information such as usernames, passwords, and credit card details. These attacks are most commonly initiated through malicious URLs distributed via email, SMS, or social media.

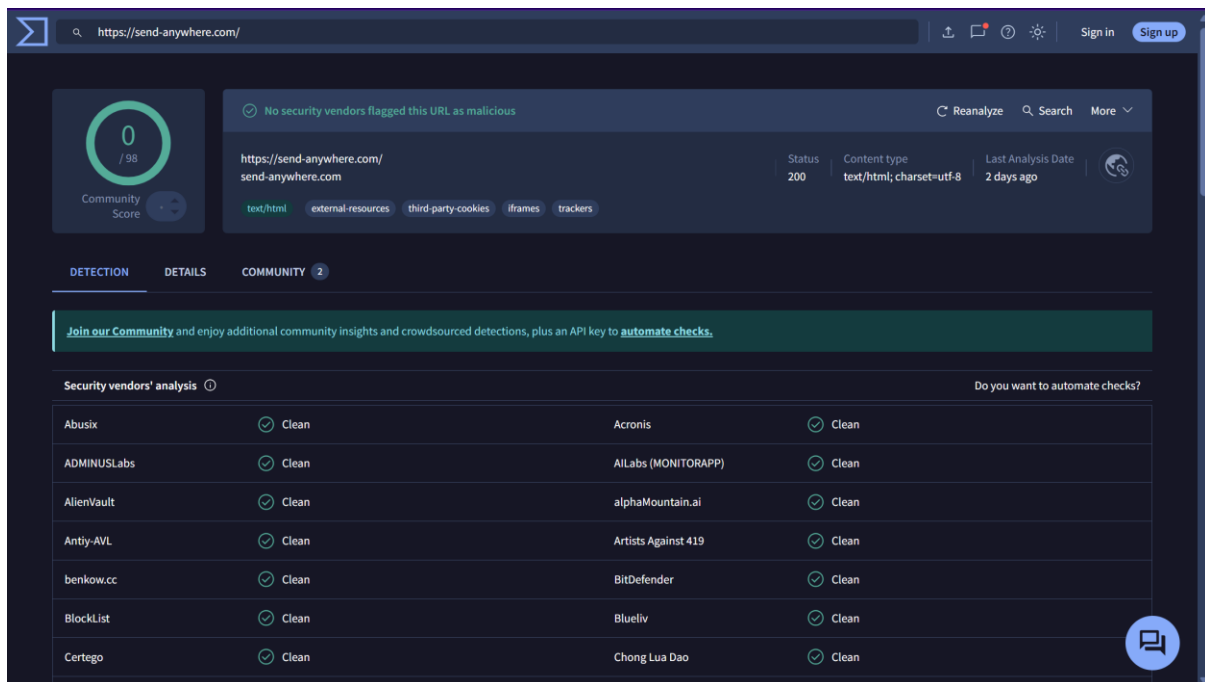
Several tools can help identify these threats:

- **Online URL Scanners:** Services like VirusTotal and Google's Safe Browsing checker analyze a URL against a vast database of known threats from various security vendors. They can identify URLs associated with malware, phishing, and other malicious activities.
- **Browser Extensions:** Security extensions integrate directly into the web browser to provide real-time protection. They often work by checking visited URLs against a blacklist, analyzing the structure of the URL for suspicious patterns, and in some cases, using machine learning to detect new threats.

Procedure:



Observations:



The screenshot shows the VirusShare website interface for the URL `https://send-anywhere.com/`. The top navigation bar includes a search icon, the URL, and links for 'Sign in' and 'Sign up'. Below the navigation bar, a green banner indicates 'No security vendors flagged this URL as malicious'. The main content area displays the URL, status (200), content type (text/html; charset=utf-8), and last analysis date (2 days ago). A 'Community Score' of 0/98 is shown on the left. Below the main content, there are tabs for 'DETECTION', 'DETAILS', and 'COMMUNITY'. The 'COMMUNITY' tab is active, showing a table of security vendors' analysis results. The table lists 16 vendors, all of whom have flagged the URL as 'Clean'. A 'Do you want to automate checks?' button is located at the bottom right of the table.

Security vendors' analysis		Do you want to automate checks?	
Abusix	Clean	Acronis	Clean
ADMINUSLabs	Clean	AILabs (MONITORAPP)	Clean
AlienVault	Clean	alphaMountain.ai	Clean
Antiy-AVL	Clean	Artists Against 419	Clean
benkow.cc	Clean	BitDefender	Clean
BlockList	Clean	Blueliv	Clean
Certego	Clean	Chong Lua Dao	Clean

Result:

The combination of online URL scanners and browser-based security tools provides an effective, multi-layered defense against phishing attacks. Online scanners are excellent for pre-emptively checking suspicious links, while browser extensions offer real-time protection during active web browsing. This experiment confirms that these tools can successfully identify and block access to known phishing domains, significantly reducing the risk of credential theft and other security breaches.

Precautions:

- **Do Not Enter Information:** When testing a potentially live phishing site, never enter any real personal information, usernames, or passwords.
- **Use a Controlled Environment:** For advanced analysis, it is best to use a virtual machine or an isolated system to prevent any potential malware from affecting your primary computer.
- **Trust Reputable Sources:** Only download browser extensions from official web stores (Chrome Web Store, Firefox Add-ons, etc.) and check their ratings and reviews.
- **No Tool is Perfect:** While these tools are highly effective, new phishing sites are created constantly. Always maintain a healthy level of skepticism and practice safe browsing habits.

EXPERIMENT - 10

Aim: To capture an API call containing a Bearer JWT and replay it to access the API. Learn token lifetime behaviour.

Material Required:

- A web proxy tool (e.g., Burp Suite Community Edition, OWASP ZAP).
- An API client (e.g., Postman, Insomnia).

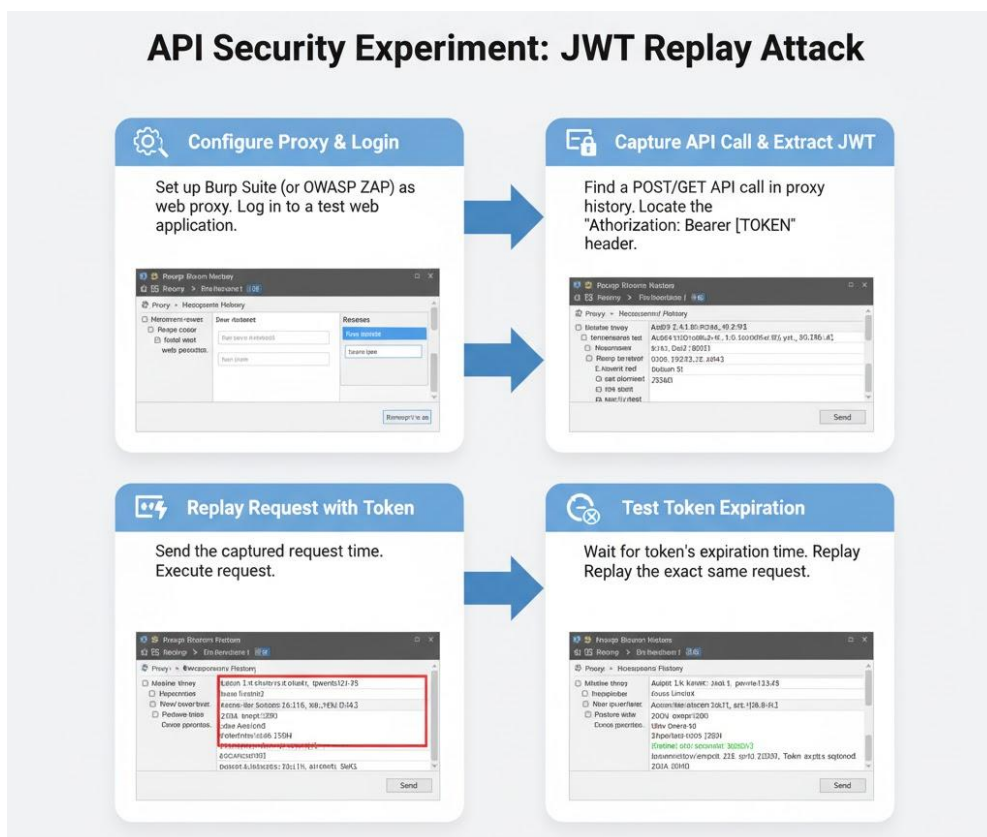
Theory:

JSON Web Tokens (JWTs) are a standard method for securely representing claims between two parties. In web applications, they are often used for authentication and authorization. Once a user logs in, the server issues a JWT, which the client then sends back with every subsequent API request to prove its identity.

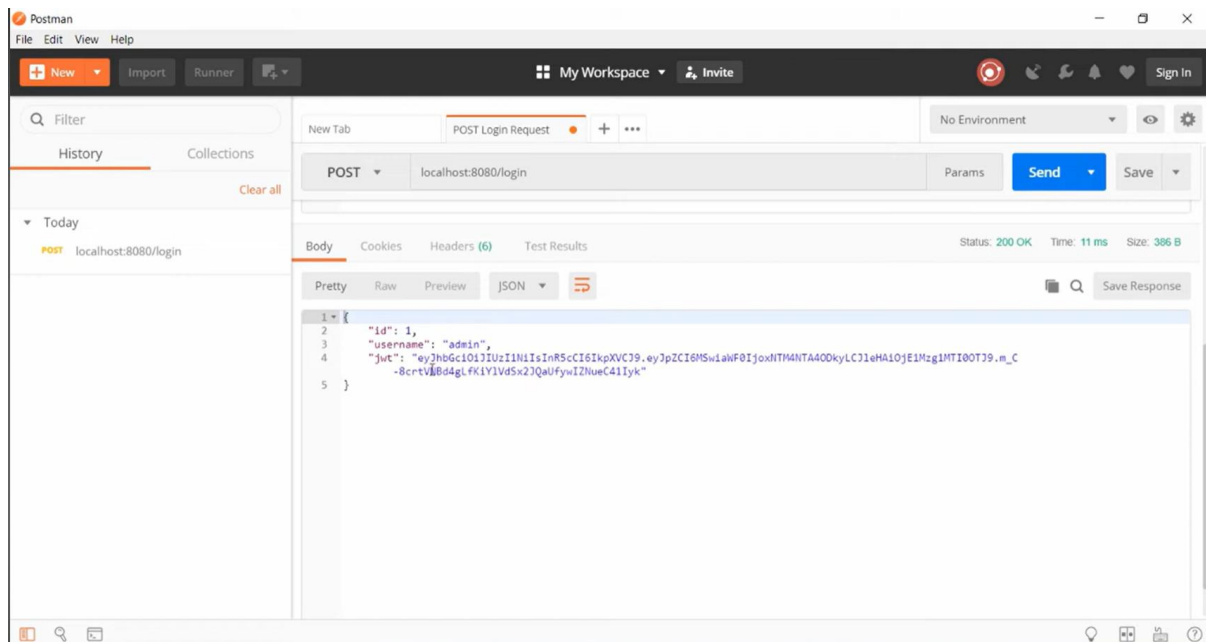
A **Bearer Token** is a type of access token. The "bearer" (the one holding the token) can use it to access protected resources without needing to prove possession of a cryptographic key. This means that if an attacker intercepts a valid JWT Bearer token, they can use it to impersonate the legitimate user until that token expires.

This experiment demonstrates a **replay attack**, a common security concern where an attacker intercepts a valid data transmission (like an API call with a token) and maliciously re-sends it to carry out an unauthorized action.

Procedure:



Observations:



Result:

This experiment successfully demonstrates that a valid JWT Bearer token can be captured and replayed by another party to gain unauthorized access to an API. The key takeaway is that the **security of a Bearer token relies heavily on its confidentiality and its limited lifetime**. Anyone who possesses the token within its validity period can use it. The primary server-side mitigation against long-term replay attacks is enforcing short, reasonable token expiration times.

Precautions:

- **Ethical Use Only:** This procedure should only be performed on applications and systems you own or have explicit, written permission to test. Unauthorized interception of data is illegal.
- **Always Use HTTPS:** This type of attack is much easier on unencrypted HTTP. Always use HTTPS (TLS) to encrypt the entire communication channel, making it significantly harder for an attacker to intercept the token in the first place.
- **Secure Token Storage:** On the client-side, tokens should be stored securely. Using HttpOnly cookies is generally safer than storing them in localStorage, as it prevents access from client-side scripts.
- **Implement Token Revocation:** For high-security applications, implement a mechanism to revoke tokens on the server-side (e.g., upon user logout), so they cannot be used even if they haven't expired.